# 04 · Team split (4 parts)

Design v5 is frozen. The P0 scope in 02-build-plan.md is split across four people. Each part owns its folders and its tests. The parts only talk to each other through the interfaces in §2.

Pick a part, write your name in the table, and read only your part's sections and its listed architecture docs.

| Part | Area | Owner |
|---|---|---|
| **Part 1** | Agent loop + token/accuracy engine | Atul |
| **Part 2** | Gateway: Bifrost, failover, budget, local routing, Tavily | _ |
| **Part 3** | Security + approvals + confidential detection | _ |
| **Part 4** | Benchmark, red-team, UX, submission | _ |

## 1. Repo layout and ownership

```
crocodile/
  cli.py                Part 1 (Typer entry: run; other parts add their own subcommands)
  loop.py state.py prompt.py              Part 1
  tools/ read.py search.py list.py edit.py tests.py      Part 1 (tests.py consent check calls Part 3)
  engine/ gate.py readcache.py masking.py ladder.py thrash.py   Part 1
  gateway/ egress.py fallback.py journal.py budget.py local.py tavily.py prices.py   Part 2
  security/ approvals.py cmd.py git.py paths.py secrets.py undo.py integrity.py confidential.py audit.py
Part 3
  ui/ quote.py status.py receipt.py      Part 4
  contracts.py          shared; changes need two owners to agree
bifrost/config.json     Part 2
redteam/                Part 4 (tests), Part 3 (reviews expected outcomes)
bench/ race.py report.py prices.yaml   Part 4
scripts/ prep.ps1       Part 4
.github/workflows/      Part 4
```

Rule: **don't edit another part's folder.** If you need a change there, ask its owner, or add a TODO in `contracts.py` and tell them.

## 2. Interfaces (agree on these first)

All of them live in `crocodile/contracts.py`. Each owner merges a **stub** first, so nobody waits on anyone else.

| Interface | Owner | Used by | Stub behaviour |
|---|---|---|---|
| `egress.chat(messages, tools, *, session, tier) -> ChatResult` (text, tool_calls, usage incl. cached, model_used, is_fallback, error_class) | Part 2 | Part 1 | Calls one model directly with no budget check |
| `egress.reserve(session, est_tokens) -> Reservation` / `reconcile(res, usage)` | Part 2 | Part 1 (called by `chat` internally) | Always allows |
| `journal.append(session, event: dict)` / `journal.resume(session) -> State` | Part 2 | Part 1, 3 | Appends JSONL |
| `approvals.ask(action: Action) -> Decision` (Action = kind write/shell/git/test_consent, argv or diff, source `file:line` or None) | Part 3 | Part 1 tools | Console y/n |

| Interface | Owner | Used by | Stub behaviour |
|---|---|---|---|
| `confidential.mode_for(repo_path) -> Mode` ("public" or "confidential") | Part 3 | Part 2 (local routing + Tavily off) | Returns `"public"` |
| `cmd.run(argv, cwd, *, timeout) -> CmdResult` (argv only, hardened env) | Part 3 | Part 1 (`run_cmd`, `run_tests`) | `subprocess.run(argv, shell=False)` |
| `crocodile run --repo X --task-file T --json-out R.json` (exit 0 = solved claim; R has tokens, cost, steps, models, fallbacks) | Part 1 | Part 4 | — |
| Event names in the journal (`model_call`, `tool_started`, `tool_done`, `approval`, `takeover`, `budget_warn`, `budget_stop`) | Part 2 | everyone | — |

## 3. The parts

Each part has 6 build items and about the same amount of work. Part 1 is still the most critical (everyone waits on `crocodile run`), so it has the fewest extras.

| Part | Items | Early load | Late load | Mandatory rule it owns |
|---|---|---|---|---|
| 1 | 6 | Heavy | Medium | (token results) |
| 2 | 6 | Heavy | Medium | Failover notice, pre-call budget, local-only routing |
| 3 | 6 | Medium | Medium | Approvals, confidential detection |
| 4 | 6 + leads final stretch | Medium | Heavy, shared by all | (proof) |

### Part 1: Agent loop + token/accuracy engine

Read: architecture/02, 03, 08.

1. The ~400-line loop: tools `read`, `search`, `list`, `edit` (staged SEARCH/REPLACE matcher), `run_tests`, `run_cmd`, git read templates; step cap 30; pinned task state.
2. `crocodile run` with `--json-out` (Part 4 is blocked until this works, so ship it first).
3. Output gate (head + tail + failures, spill to disk; calls Part 3's secret scan).
4. Read cache ("unchanged"/diff), cache-stable prefix + frozen repo map.
5. Verification ladder (syntax → ruff isolated → affected tests → full), stop-on-green with oracle freeze, thrash detector, cheap-first escalation.
6. Batched masking of old tool output. Every lever sits **behind a flag** so Part 4 can run the "all-off twin".

Done when: one Polyglot task is solved end to end through the real `egress`, and each flag toggles.

### Part 2: Gateway: Bifrost, failover, budget, local routing, Tavily

Read: architecture/06, 07 (routing parts), 09, 04 §7, ADR-001, ADR-005.

1. **First thing:** start Bifrost v2.5.1 locally, write `bifrost/config.json`, create the VKs, run day-0 checks 1–6 and 9 (02 §6), and post the results in the team chat. Everyone depends on this.
2. `egress.chat`: prefixed model ids, explicit `fallbacks`, routing headers, and a **MODEL TAKEOVER** banner showing the error class. Client-side retry if the Bifrost fallback check fails.
3. Journal (JSONL) + idempotent resume (`tool_started` handling).
4. Budget: atomic SQLite reservation priced at the most expensive model in the chain, lease, reconcile, admin-only `limits.yaml`, 80% warning, ladder strong → cheap → local → pause.
5. Local-only routing when `mode_for` says confidential: local-only VK, `ollama/` ids with `fallbacks: []`, local model allowlist (reject `:cloud`/`-cloud`, non-loopback `OLLAMA_HOST`), fit to `OLLAMA_CONTEXT_LENGTH`, egress log.
6. Tavily `web_docs` tool (lean params, gated, unregistered when confidential); `prices.yaml` shared with Part 4.

Done when: a bogus primary key shows the banner with `auth`; a confidential repo only ever produces `ollama/*` calls.

### Part 3: Security + approvals

Read: architecture/05, 07 (detection + audit), ADR-003, ADR-004, ADR-006.

1. `approvals.ask` on the console (CONIN$, escaping), the red "THIS EXECUTES CODE FROM THE REPO" test-consent panel, and the exec-surface re-prompt.
2. `cmd.run`: argv only, Windows path/exe rules (never resolve from inside the repo); hardened git with a per-call config check and rev validation.
3. Provenance-lite: a red prompt naming `file:line` when an action came from repo text; secret scan used by the output gate.
4. Confidential detection: `mode_for` (realpaths up + bounded down), plus `audit` and the canary check over Part 2's egress log.
5. Shadow-git file undo.
6. State integrity (HMAC + watch-set) and user `doctor`.

Done when: the red `file:line` prompt shows on the README fixture; tampering with a state file stops the agent; Part 4's red-team cases for these pass.

### Part 4: Benchmark, red-team, UX, submission

Read: architecture/10, 11, 01-playbook, 03-claims.

1. WSL2 + mini-swe-agent 2.4.6 pointed at Bifrost; day-0 check 10 (one task, usage logged). If it can't be made to work, the baseline becomes the all-off twin only, and the pitch says so.
2. Bench harness: copy tasks without `.meta/`, identical task text, caps, restore tests before grading, JSONL output, cost from Bifrost logs × `prices.yaml`. Polyglot 34 + QuixBugs 20. `bench --limit 3` as soon as `crocodile run` lands, then a 10-task smoke.
3. Red-team MVP: 15 cases in `redteam/` as pytest tests, from the list in architecture/10 §3. Part 3 reviews the expected outcomes.
4. `bench race` (8 processes, ours vs naive) + `test_concurrency_50` + `test_tier_reserve` against Part 2's budget.
5. User-facing output: static `quote`, status line, receipt (all read the journal); CI (windows + ubuntu).
6. `report.py`: paired diff, bootstrap CI, CPST, cost ratio, Pareto chart.

Done when: `bench/report.md` exists from real runs, RT 15/15 pass in CI, and the race shows $0.0000 overspend for us.

### Final stretch (shared, after feature freeze)

Part 4 leads; everyone records the clip for what they built.

- Part 1: demo repos + the token-savings clip.
- Part 2: failover banner + budget-stop clips.
- Part 3: approval/red-prompt + confidential clips.
- Part 4: `scripts/prep.ps1`, video edit, README from the frozen numbers (only wording from 03-claims), AI-use disclosure, submission form.

## 4. Shared milestones (in order)

1. `contracts.py` + all stubs merged; Bifrost running (all; Part 2).
2. Day-0 checks posted; mini-swe-agent go/no-go (Part 2, Part 4).
3. `crocodile run --json-out` works on stubs (Part 1 → Part 4).
4. **Integration:** one task through real Bifrost with approvals + budget on (all).
5. Bench run 1 started: 54 tasks × ours + mini (Part 4).
6. Fix only crash bugs from run 1; start run 2 + all-off twin (all → Part 4).

7. **Feature freeze.** Unfinished work goes on the roadmap slide (all).
8. **Numbers frozen.** Video clips recorded (all, Part 4 leads).
9. Submitted (Part 4).

If a part slips, cut from the bottom of its list. Never cut the four mandatory rules: failover notice (Part 2), pre-call budget (Part 2), confidential local-only (Part 2 routing + Part 3 detection), approvals (Part 3).

## 5. Working rules

- Branch per part (`part1/...` to `part4/...`), small PRs to `main`, merge often. Run `pytest` + `ruff` before merging.
- Commit under your own name. No AI co-author lines or "generated with" footers in commits or PRs; AI use is disclosed once, in the submission.
- No secrets in the repo. Keys go in `.env` (git-ignored) or Windows Credential Manager.
- Public material never names peer teams (`docs/internal/`, research 02/06/08/09 stay internal).
- When using an AI assistant, give it this file + your part's architecture docs only, not the whole repo.